Self-Organizing Octonionic Tries: Algebraic Memory Without Gradient Descent

Antonio Escalera

March 2026

Working Draft

Abstract

We propose a self-organizing hierarchical memory structure in which octonionic algebra replaces gradient-based learning entirely. The structure, an **octonionic trie**, is a dynamically growing tree whose nodes are individual octonions. Routing through the trie is governed by Fano plane subalgebra decomposition, growth is triggered by the associator (a measure of algebraic incompatibility), updates are performed by octonionic composition, and consistency is verified via algebraic inversion. The trie monitors its own structural health through invariants derived from the algebra: compression efficiency, subalgebra decomposition cleanliness, composition error bounds, and prediction consistency.

The result is a memory system that organizes, grows, consolidates, and self-corrects using only the algebraic properties of the octonions, without weight matrices, loss functions, or backward passes. The octonionic algebra provides the routing structure (7 quaternionic subalgebras via the Fano plane), the novelty detection mechanism (the associator), the update rule (norm-preserving multiplication), the consistency check (algebraic inversion), and the structural health metrics (associator norms, composition error). These are typically five separate engineered components in conventional memory-augmented architectures; here they emerge from a single algebraic substrate.

This work builds on the mathematical foundations established in the companion thesis on octonionic neural networks [Escalera, 2026], extending the algebraic framework from gradient-trained networks to self-organizing structures that operate without optimization.

Contents

1 Introduction and Motivation **3**
1.1 Relationship to Companion Work . . . 3

2 The Associator as a Computational Primitive **3**
2.1 Definition and Algebraic Properties . . . 4
2.2 Computational Interpretation . . . 4
2.3 The Associator vs. Inner Product . . . 4

3 Architecture of the Octonionic Trie **5**
3.1 Structure . . . 5
3.2 Encoding: From Raw Data to Octonions . . . 5
3.2.1 Hierarchical Projection . . . 6
3.3 Core Operations . . . 6
3.3.1 Compose: Routing and Update . . . 6
3.3.2 Detect: Novelty and Branching . . . 7
3.3.3 Ruminate: Consistency Verification . . . 7
3.3.4 Consolidate: Pruning and Merging . . . 7

3.4 Algebraic Residuals . . . 7
3.5 Memory Buffer and Relevance . . . 8

4 Structural Invariants **8**
4.1 Composition Error Bound . . . 8
4.2 Compression Efficiency . . . 9
4.3 Subalgebra Decomposition Cleanliness . . . 9
4.4 Prediction Consistency . . . 9
4.5 Associator Health . . . 9

5 Properties of the Architecture **9**
5.1 No Gradient Computation Required . . . 9
5.2 Encoder Invariance of Trie Structure . . . 10
5.3 Information Preservation . . . 10
5.4 Natural Hierarchy . . . 10

6 Relationship to Existing Architectures **10**

7 Open Questions **11**
7.1 Empirical Validation Prerequisites . . . 11
7.2 Geometric Embedding of the Trie . . . 11
7.3 Composition Order . . . 11
7.4 Adaptive Thresholds . . . 12
7.5 Relevance Metric for Memory Buffer . . . 12

8 Conclusion **12**

1 Introduction and Motivation

Memory-augmented neural architectures (Neural Turing Machines [Graves et al., 2014], Differentiable Neural Computers [Graves et al., 2016], Memory Networks [Weston et al., 2015]) address a fundamental limitation of feedforward networks: the inability to store and retrieve structured information beyond what is captured in fixed weight matrices. These architectures introduce external memory modules with learned read/write controllers, enabling the network to maintain and manipulate an explicit memory state.

However, existing memory architectures share a common design pattern: the memory structure (flat array, matrix, or graph) is fixed a priori, and a trained controller learns to use it. The controller is optimized via gradient descent, the memory layout is predetermined, and the system cannot grow or restructure its memory in response to the data it encounters.

This paper proposes a fundamentally different approach. Rather than training a controller to manage a fixed memory, we construct a memory that *organizes itself* using the algebraic properties of the octonions. The memory is a tree (trie) whose nodes are individual octonions. The tree grows when it encounters novel structure, updates by algebraic composition, verifies its own consistency via inversion, and prunes itself when structure becomes redundant. No component of this system requires gradient-based optimization.

The key insight is that the octonionic algebra provides, through its intrinsic mathematical structure, all the mechanisms that memory-augmented architectures typically engineer separately:

1. **Routing** via Fano plane subalgebra decomposition (7 quaternionic channels).
2. **Novelty detection** via the associator $[a, b, c] = (ab)c - a(bc)$.
3. **Content update** via norm-preserving octonionic multiplication.
4. **Consistency verification** via algebraic inversion ($x^{-1} = \bar{x}/|x|^2$).
5. **Structural health monitoring** via associator norms, composition error, and subalgebra coherence.

These five mechanisms are not independent modules bolted together; they are consequences of the same algebraic structure. This economy of mechanism is the central argument for the octonionic trie.

1.1 Relationship to Companion Work

This paper depends on and extends the foundations established in Escalera [2026]:

- **Algebraic foundations** (Sections 1–3 of the companion): Hurwitz's theorem, octonionic multiplication, the Fano plane, G_2 automorphisms, the density argument.
- **Reversibility thesis** (Section 2): Algebraic invertibility enabling backward reasoning. The octonionic trie uses inversion for consistency checking ("rumination").
- **Numerical stability** (Phase 4 experimental results): Composition error bounds that determine the trie's maximum depth.
- **Associator and subalgebra analysis** (Phase 9): Whether the associator carries useful information and whether subalgebras specialize. These are prerequisites for the trie's routing and novelty detection mechanisms.

The companion thesis validates these properties for gradient-trained octonionic networks. This paper proposes that the same properties support a qualitatively different computational paradigm: self-organization without optimization.

2 The Associator as a Computational Primitive

The associator is the mathematical object that makes the octonionic trie possible. This section establishes its properties and computational interpretation in detail.

2.1 Definition and Algebraic Properties

Definition 2.1 (Associator)**.** *For three octonions $a, b, c \in \mathbb{O}$, the* ***associator*** *is defined as:*

$$[a, b, c] = (ab)c - a(bc). \tag{1}$$

The associator is an octonion-valued trilinear form measuring the failure of associativity for a given triple.

The associator satisfies the following properties:

1. **Total antisymmetry:** $[a, b, c] = -[b, a, c] = -[a, c, b] = -[c, b, a]$. Swapping any two arguments flips the sign. This means the associator is sensitive to the ordering of all three elements.
2. **Alternativity:** $[a, a, b] = [a, b, b] = 0$ for all $a, b \in \mathbb{O}$. When any two arguments are equal, the associator vanishes. The algebra is "alternative" even though it is not associative.
3. **Subalgebra characterization:** $[a, b, c] = 0$ if and only if a, b, and c generate an associative subalgebra (i.e., they all lie within the same quaternionic subalgebra of $\mathbb{O}$, or within $\mathbb{R}$ or $\mathbb{C}$). A nonzero associator certifies that the three elements span structure across multiple subalgebras.
4. **Continuity:** The associator is a continuous function of its arguments. Small perturbations of the inputs produce small changes in the associator. This means the transition from "compatible" (small associator) to "incompatible" (large associator) is gradual, not discontinuous.
5. **Norm bound:** $|[a, b, c]| \leq 2|a||b||c|$. The associator norm is bounded by the product of the input norms. For unit octonions, the maximum associator norm is 2.

2.2 Computational Interpretation

In the context of the octonionic trie, the associator answers a specific question: *given a new input x, an existing child node c, and their parent node n, does x belong in the same branch as c?*

- $|[x, c, n]| \approx 0$: The triple (x, c, n) generates an associative subalgebra. The input x is algebraically compatible with the existing child c in the context of parent n. Grouping does not matter; x can be composed with c without ambiguity. **Route to c and update.**
- $|[x, c, n]| \gg 0$: The triple spans multiple subalgebras. The input x introduces structure that is algebraically incompatible with c in the context of n. The order of composition matters substantially, meaning x carries information that c's branch cannot accommodate without distortion. x **does not belong in this branch.**
- $|[x, c_i, n]| \gg 0$ for *all* children c_i: The input is incompatible with every existing branch. It represents genuinely novel structure. **Candidate for new node creation.**

The associator norm is a continuous scalar, providing a graded signal rather than a binary decision. The threshold separating "compatible" from "incompatible" is a design parameter that may be adaptive (e.g., proportional to the mean associator norm observed at that node).

2.3 The Associator vs. Inner Product

A natural question is why the associator is preferable to simpler similarity measures (e.g., octonionic inner product $\langle x, c\rangle = \mathrm{Re}(\bar{x}c)$) for routing decisions.

The inner product measures how aligned two octonions are as vectors in $\mathbb{R}^8$. It is a pairwise measure that ignores the algebraic context in which the comparison occurs. The associator is a *three-way* measure that captures the relationship between two elements *in the context of a third* (the parent node). This context-sensitivity is precisely what routing in a trie requires: whether an input belongs with a child depends not just on the input-child similarity, but on how both relate to the parent.

Furthermore, the inner product is symmetric and exists in every algebra (including $\mathbb{R}^8$ with no octonionic structure). The associator is antisymmetric, trilinear, and exists *only* in non-associative algebras. It is the uniquely octonionic signal: the information that octonions carry and that quaternions, complex numbers, and reals cannot.

3 Architecture of the Octonionic Trie

3.1 Structure

Definition 3.1 (Octonionic Trie)**.** *An* ***octonionic trie*** $\mathcal{T}$ *is a rooted tree in which:*

- *Each node v stores a single octonion $o_v \in \mathbb{O}$.*
- *Each node has at most 7 children, corresponding to the 7 quaternionic subalgebras of $\mathbb{O}$ defined by the Fano plane.*
- *The tree grows dynamically: nodes are created when new data is algebraically incompatible with all existing branches, and consolidated when branches become redundant.*

The branching factor of 7 is not a design choice; it is a consequence of the Fano plane structure. The octonion $\mathbb{O}$ contains exactly 7 quaternionic subalgebras, each defined by an oriented triple (e_i, e_j, e_k) of imaginary basis units. These subalgebras overlap (each imaginary unit belongs to exactly 3 subalgebras), providing a rich, interconnected routing structure rather than a disjoint partition.

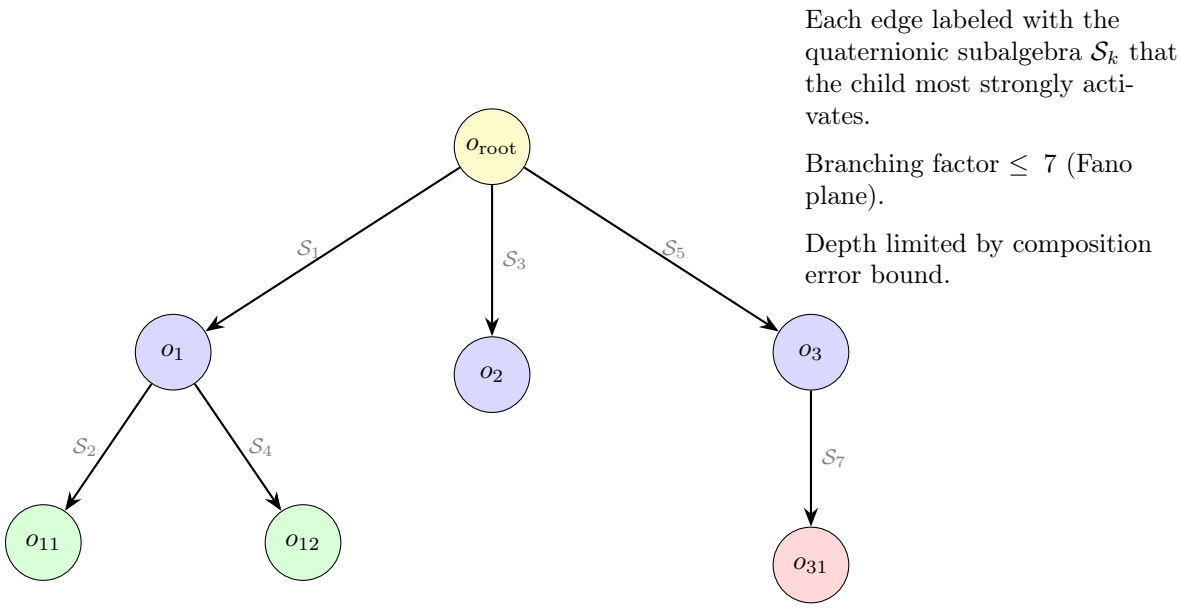

Figure 1: An octonionic trie with three levels. Each node stores a single octonion. Edges are labeled with the subalgebra activated by the child relative to its parent. Not all 7 children need to exist at each node; the trie grows on demand.

3.2 Encoding: From Raw Data to Octonions

The octonionic trie operates on octonionic inputs. Raw data must be encoded into $\mathbb{O}$ before entering the trie. A key architectural property is that **the encoder is fully decoupled from the trie.**

The Fano plane subalgebra structure is a property of the octonionic multiplication table (the structure constants tensor C_{ijk}), not of the data or the encoding. When two octonions are multiplied and the result is decomposed along subalgebras, the decomposition is determined

by C_{ijk} regardless of how the octonions were produced. Consequently, the trie's self-organizing behavior is invariant to encoder quality: a better encoder produces richer octonionic representations, enabling the trie to capture more meaningful structure, but the trie's routing, growth, and consolidation mechanisms function identically for any encoder.

This decoupling has a practical consequence: any embedding model that produces dense vector representations (e.g., large language model embeddings, vision encoders, multimodal embeddings) can serve as the encoder, with a projection step mapping the embedding space into $\mathbb{O}$.

3.2.1 Hierarchical Projection

A single linear projection from a high-dimensional embedding (e.g., $\mathbb{R}^{768}$) to $\mathbb{O}$ ($\mathbb{R}^8$) would discard most structure. However, the trie's tree structure provides a natural solution: **hierarchical decomposition.**

Rather than encoding the entire embedding into a single octonion, the encoding is distributed across trie levels:

- **Level 0:** A projection captures the coarsest structure of the embedding (e.g., the top 8 principal components). This determines the root-level routing.
- **Level k:** A projection captures the next level of detail, conditioned on the routing decisions at levels $0, \ldots, k-1$.
- **Cumulative representation:** A path of depth k through the trie captures $8k$ effective dimensions of the original embedding, distributed across k nodes.

The projections can be derived from principal component analysis of the embedding space: principal components 1–8 map to Level 0, components 9–16 to Level 1, and so on. This assignment is deterministic, requires no training, and naturally places the highest-variance (most discriminative) dimensions at the shallowest trie levels.

The trie's depth limit (determined by the composition error invariant, section 4) then corresponds to the maximum number of principal components the system can faithfully represent. This provides a principled, data-dependent capacity bound.

Epistemic honesty note. *Whether PCA-based hierarchical projection preserves sufficient structure for routing to produce semantically meaningful trie organization is an empirical question. Alternative decompositions (independent component analysis, learned hierarchical projections) may perform better for specific data domains. The architectural claim is that the trie's self-organization is invariant to the choice of projection; the quality of the resulting organization is not.*

3.3 Core Operations

The octonionic trie supports four operations. None requires gradient computation.

3.3.1 Compose: Routing and Update

When a new input $x \in \mathbb{O}$ arrives, it is routed through the trie from root to leaf:

1. At each node n with children $\{c_1, \ldots, c_k\}$, compute the octonionic product $p = n \cdot x$.
2. Decompose p along the 7 quaternionic subalgebras $\mathcal{S}_1, \ldots, \mathcal{S}_7$ of $\mathbb{O}$.
3. Select the child whose subalgebra has the largest projection: $c^* = \arg\max_i |\mathrm{proj}_{\mathcal{S}_i}(p)|$.
4. Compute the associator $[x, c^*, n]$. If the norm is below the compatibility threshold, descend into c^* and repeat.
5. At the leaf, update the node by composition: $o'_{\text{leaf}} = o_{\text{leaf}} \cdot x$.

The update step is norm-preserving when both operands are unit octonions ($|ab| = |a||b|$). Sequential compositions do not cause the representation to explode or vanish.

The components of p that fall outside the selected subalgebra constitute the **algebraic residual**. This residual is not discarded; it is recorded as metadata at the node (section 3.4).

3.3.2 Detect: Novelty and Branching

If the associator $[x, c_i, n]$ exceeds the compatibility threshold for *all* existing children at a node, the input is flagged as a branching candidate. A branching event is not triggered immediately; persistent incompatibility across multiple inputs at the same node is required.

When a branching event is confirmed:

1. A new child node is created at the node n.
2. The new child is initialized with the input x (or a composition of the accumulated incompatible inputs).
3. The new child is assigned to the subalgebra with the strongest residual accumulation (section 3.4), ensuring it occupies the region of octonionic space where the most unaccommodated structure has been observed.

3.3.3 Ruminate: Consistency Verification

Before committing an update (composing x into a leaf node), the trie verifies that the update is consistent with prior predictions that routed through the affected branch.

Each node maintains a bounded buffer of prior predictions (section 3.5). The rumination procedure:

1. Tentatively compute the updated representation: $o' = o \cdot x$.
2. For each buffered prediction y that routed through this node, use algebraic inversion to estimate the counterfactual: "If this node had been o' instead of o, what would the prediction have been?"
3. Compute the discrepancy between the original prediction and the counterfactual.
4. If the maximum discrepancy exceeds the consistency threshold: reject the update and escalate to a branching decision. The input introduces structure that is inconsistent with the node's history.
5. If all discrepancies are within threshold: commit the update.

This is counterfactual reasoning enabled by algebraic invertibility. The system asks "if I change my understanding here, do my past conclusions still hold?" and acts on the answer.

3.3.4 Consolidate: Pruning and Merging

The trie periodically evaluates its structural health and consolidates:

- **Sibling absorption:** If a node has not been routed to recently (below a recency threshold), its representation is composed into its nearest sibling: $o'_{\text{sibling}} = o_{\text{sibling}} \cdot o_{\text{unused}}$. The unused node is removed. Because composition is invertible, the absorbed information is not destroyed but integrated into a more general representation.
- **Child merging:** If two children at a node have small pairwise associator ($|[c_i, c_j, n]| < \epsilon$), they are too similar to warrant separate branches. One is composed into the other and removed.
- **Depth limiting:** If the composition error at a given depth exceeds the scaling threshold (section 4), the node stops accepting new children. This branch has reached its representational resolution limit.

3.4 Algebraic Residuals

When an input is routed to a specific child via subalgebra decomposition, the components of the octonionic product that fall in other subalgebras are the **algebraic residual**. These are not discarded but serve as metadata:

- A running mean direction (unit octonion) is maintained per unoccupied subalgebra at each node.
- A count of inputs contributing to each residual direction is tracked.
- When a branching event occurs (section 3.3), the residual history informs the orientation of the new child: the subalgebra with the strongest accumulated residual is the natural location for the new branch.

Residuals are informational, not decisional. They record "what structure has been observed but not accommodated" without triggering any action. They answer the question "if we ever need to branch here, where should the new child go?" The associator remains the sole trigger for branching decisions.

3.5 Memory Buffer and Relevance

Each node (or each branch, defined as the path from root to a given node) maintains a bounded buffer of prior predictions for use in rumination (section 3.3).

The buffer operates as follows:

- **Capacity:** Fixed-size ring buffer. Recent predictions are always retained.
- **Relevance filtering:** When an input arrives at a node, buffered predictions are scored for relevance. The relevance metric (octonionic inner product, norm-based similarity, or other measures) determines which memories are consulted during rumination. Predictions below a relevance threshold are skipped, bounding computation.
- **Percolation:** When a leaf makes a prediction, it is added to the buffers of all ancestor nodes along its branch. This ensures that consistency checks at any level of the trie have access to predictions from the entire subtree.

This memory model mirrors aspects of human memory: recent events are immediately accessible (ring buffer), and strongly relevant past experiences surface during familiar situations (relevance filtering). The octonionic algebra provides a natural relevance metric through the inner product $\langle x, y\rangle = \mathrm{Re}(\bar{x}y)$, though whether this is the optimal metric for relevance is an empirical question.

4 Structural Invariants

The octonionic trie monitors its own structural health through five invariants derived from the algebra. These invariants replace the role that loss functions play in gradient-trained systems.

4.1 Composition Error Bound

After composing k inputs into a single node via sequential octonionic multiplication, floating-point error accumulates. The trie tracks:

$$\varepsilon(d) = \frac{|x^{-1} \cdot (o \cdot x) - o|}{|o|} \tag{2}$$

where o is the node's representation and x is the most recent input. When $\varepsilon(d)$ exceeds a threshold at depth d, the node stops growing deeper. This invariant establishes the trie's resolution limit per branch, connecting directly to the numerical stability characterization from Phase 4 of the companion thesis.

Different branches may have different depth limits depending on how cleanly their content decomposes along subalgebras. Branches with strong subalgebra alignment (small associators) accumulate error more slowly and can grow deeper.

4.2 Compression Efficiency

The trie monitors the ratio of structural complexity to prediction quality:

$$\rho = \frac{\text{number of nodes}}{\text{effective prediction capacity}}. \tag{3}$$

If a subtree has many nodes but contributes minimally to prediction quality (measured by how often its nodes are consulted during routing), the subtree is a candidate for consolidation. This is the minimum description length (MDL) principle applied structurally rather than as an optimization objective.

4.3 Subalgebra Decomposition Cleanliness

At each internal node, the pairwise associators between children measure how distinct the branches are:

$$\text{cleanliness}(n) = \min_{i \neq j} |[c_i, c_j, n]|. \tag{4}$$

High cleanliness means the children occupy genuinely different subalgebras and the branching is justified. Low cleanliness means the children are algebraically similar and should be considered for merging.

4.4 Prediction Consistency

The maximum discrepancy observed during rumination across the memory buffer:

$$\delta(n) = \max_{y \in \text{buffer}(n)} |\text{counterfactual}(y, o') - \text{original}(y)|. \tag{5}$$

When $\delta(n)$ is persistently high, the node's representation is unstable (frequent updates that would invalidate prior predictions). This may indicate the node is attempting to represent fundamentally incompatible information and should be split.

4.5 Associator Health

The mean associator norm at each node, averaged over recent inputs:

$$\alpha(n) = \frac{1}{K} \sum_{k=1}^{K} |[x_k, c_k^*, n]|, \tag{6}$$

where c_k^* is the selected child for input x_k. A persistently high $\alpha(n)$ means the node's children are not accommodating incoming data well. A persistently low $\alpha(n)$ means the routing structure is effective. Trends in $\alpha(n)$ can signal that the node needs restructuring before a branching event is triggered.

5 Properties of the Architecture

5.1 No Gradient Computation Required

The octonionic trie operates entirely through algebraic operations: multiplication, inversion, subalgebra projection, and associator computation. None of these requires differentiability or a backward pass. The trie's "learning" consists of:

- **Composition:** accumulating structure via octonionic products.
- **Growth:** creating nodes when the associator signals novelty.
- **Consolidation:** merging or absorbing nodes when invariants indicate redundancy.
- **Self-correction:** rejecting updates that fail consistency checks.

The only component that may require gradient-based training is the encoder that maps raw data into octonionic representations (section 3.2). Once data is in $\mathbb{O}$, the trie is self-organizing.

5.2 Encoder Invariance of Trie Structure

The trie's routing, growth, and consolidation mechanisms depend on the octonionic multiplication structure (the structure constants C_{ijk}), not on the encoder. The Fano plane subalgebras, the associator, and the composition rule are properties of $\mathbb{O}$ that apply identically to any octonionic input regardless of its provenance.

A consequence: the same trie architecture can be used with different encoders (text embeddings, vision features, sensor data) without modification. The encoder determines the semantic quality of the trie's organization; the algebra determines the organizational mechanism.

5.3 Information Preservation

Octonionic composition is norm-preserving ($|ab| = |a||b|$) and invertible (a^{-1} exists for all $a \neq 0$). In exact arithmetic, no information is destroyed by composition, and any composition can be undone. Consolidation (sibling absorption) composes rather than discards, preserving information at a coarser level of representation.

In finite-precision arithmetic, composition error accumulates (section 4), providing a natural resolution limit. The trie degrades gracefully: deeper nodes have progressively lower fidelity, and the scaling invariant prevents growth beyond the point where fidelity is inadequate.

5.4 Natural Hierarchy

The trie's tree structure encodes hierarchy explicitly: parent nodes represent more general concepts, leaf nodes represent specific instances. The depth at which an input is stored reflects its specificity: inputs that are resolved at shallow depths are general (they fit cleanly into coarse-grained branches), while inputs requiring deep traversal are specific (they require fine-grained subalgebra distinctions).

This hierarchy emerges from the data and the algebra, not from architectural prescription. The trie does not have predefined "levels of abstraction"; abstraction levels arise from the subalgebra decomposition of the data itself.

6 Relationship to Existing Architectures

Table 1: Comparison with existing memory-augmented architectures.

Property	NTM/DNC	SDM	HTM	Oct-Trie
Memory structure	Flat array	Sparse distributed	Columnar	Tree (trie)
Growth mechanism	Fixed size	Fixed size	Fixed columns	Dynamic (associator)
Routing	Learned attention	Hamming distance	Spatial pooling	Subalgebra decomposition
Update rule	Learned write	Superposition	Hebbian	Algebraic composition
Consistency check	None	None	None	Algebraic inversion
Training	Backprop	None	Hebbian	None (algebraic)
Pruning	None	Decay	Decay	Sibling absorption
Self-monitoring	None	None	Anomaly	5 algebraic invariants

NTM = Neural Turing Machine [Graves et al., 2014]; DNC = Differentiable Neural Computer [Graves et al., 2016]; SDM = Sparse Distributed Memory [Kanerva, 1988]; HTM = Hierarchical Temporal Memory [Hawkins and Ahmad, 2016].

The octonionic trie shares individual features with existing architectures but combines them in a way that, to our knowledge, has not been proposed:

Kanerva's Sparse Distributed Memory [Kanerva, 1988] uses high-dimensional binary addresses for content-addressable storage. The octonionic trie replaces binary address matching with subalgebra decomposition, providing a richer routing signal (7 overlapping 3D subspaces vs. Hamming distance in a flat binary space).

Hierarchical Temporal Memory [Hawkins and Ahmad, 2016] uses columnar structure inspired by neocortical columns, with Hebbian-like learning. The octonionic trie replaces the columnar structure with the Fano plane and Hebbian updates with algebraic composition. Both share the principle of learning without backpropagation.

Neural Turing Machines [Graves et al., 2014] use learned read/write heads over a flat memory matrix. The octonionic trie replaces learned attention with algebraic routing and learned writes with algebraic composition, eliminating the need for gradient-based controller training. The NTM's flat memory becomes a dynamic tree.

7 Open Questions

7.1 Empirical Validation Prerequisites

Three empirical questions must be answered before the octonionic trie can be implemented:

1. **Subalgebra routing discriminability.** Does projecting an octonionic product onto the 7 Fano subalgebras produce a clean enough signal to route semantically related inputs to the same branches? This can be tested by encoding a dataset with known categories into $\mathbb{O}$, constructing products at trie nodes, and measuring whether within-category inputs consistently activate the same subalgebras.

2. **Associator as novelty signal.** Does the associator norm reliably spike when a genuinely novel category appears in a data stream? This can be tested on a sequential classification task where category boundaries are known.

3. **Composition depth vs. information retention.** After composing k inputs into a single node via sequential octonionic multiplication, how much of each input's structure is recoverable via inversion? The companion thesis's Phase 4 stability data provides bounds for layer-wise composition, but sequential data composition has different error characteristics.

7.2 Geometric Embedding of the Trie

The companion thesis investigates whether the octonionic state space should carry a hyperbolic metric (section 1). For the octonionic trie, the question takes a specific form: should the trie's nodes live on a hyperboloid H^7, in a Poincaré ball B^8, or in flat $\mathbb{O}$?

The trie's tree structure already encodes hierarchy explicitly (parent-child relationships). Adding a hyperbolic metric would make the embedding space *also* encode hierarchy, providing redundancy that may improve distance-based operations (relevance filtering, nearest-sibling identification for consolidation) but adds the re-projection problem identified in the companion thesis (Section 8.7).

Flat $\mathbb{O}$ with the norm encoding node confidence or weight is the simplest starting point. The trie structure provides hierarchy; the algebra provides routing. Whether a hyperbolic metric provides additional benefit is an empirical question that should be investigated after the flat-space trie is validated.

7.3 Composition Order

Octonionic multiplication is non-commutative: $a \cdot b \neq b \cdot a$ in general. The update rule $o' = o \cdot x$ (node composed with input) produces a different result from $o' = x \cdot o$ (input composed

with node). The choice affects subalgebra activation patterns and associator behavior. Both orderings should be characterized empirically; the correct choice may depend on whether the trie is intended to emphasize the existing representation (left-multiply by node) or the new information (left-multiply by input).

7.4 Adaptive Thresholds

The compatibility threshold for the associator, the consistency threshold for rumination, and the error threshold for depth limiting are design parameters. Whether these should be:

- Global constants,
- Per-node adaptive values (e.g., proportional to the node's mean associator norm),
- Derived from the data distribution (e.g., percentiles of observed associator norms),

is an open question with significant impact on the trie's sensitivity to novelty and its growth rate.

7.5 Relevance Metric for Memory Buffer

The choice of relevance metric for the memory buffer (section 3.5) affects which prior predictions are consulted during rumination. Candidates include the octonionic inner product $\langle x, y\rangle = \mathrm{Re}(\bar{x}y)$, the octonionic norm of the difference $|x - y|$, and subalgebra-projected similarity. The optimal metric likely depends on the task and may itself be a learnable component, though this would reintroduce a gradient-trained element into an otherwise algebraic system.

8 Conclusion

The octonionic trie demonstrates that the algebraic properties of the octonions, specifically the Fano plane subalgebra structure, the associator, norm-preserving multiplication, and algebraic invertibility, are sufficient to construct a self-organizing hierarchical memory without gradient-based optimization.

The architecture replaces five typically-engineered components (routing, novelty detection, content update, consistency verification, structural health monitoring) with consequences of a single algebraic structure. This economy of mechanism is the central argument: not that the octonionic trie outperforms existing memory architectures on benchmarks (this is an empirical question not yet tested), but that the algebraic substrate provides a coherent, minimal foundation from which all necessary memory operations emerge.

The trie's relationship to the companion thesis on octonionic neural networks [Escalera, 2026] is complementary. The companion thesis validates whether octonionic representations provide advantages in gradient-trained networks. This paper proposes that the same algebraic properties support a qualitatively different computational paradigm. If both are validated, the implication is that the octonionic algebra is not merely a useful parameterization for neural networks, but a general-purpose computational substrate for structured knowledge representation and reasoning.

References

Antonio Escalera. Octonionic neural networks: Division algebra structure for reversible geometric reasoning. Working draft, 2026.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. In *arXiv preprint arXiv:1410.5401*, 2014.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou,

et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538 (7626):471–476, 2016.

Jeff Hawkins and Subutai Ahmad. Why neurons have thousands of synapses, a theory of sequence memory in neocortex. *Frontiers in Neural Circuits*, 10, 2016.

Pentti Kanerva. *Sparse Distributed Memory.* MIT Press, 1988.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations (ICLR)*, 2015.